

DC-Analytics: Enterprise Retail Optimization Platform

Capstone Project Technical Documentation Report

Course Tracking: Wipro NGA Training Program (WIPRO_NGA-.Net_FSDAngular-FY27-C1)

Prepared By: Dibyajyoti Chakravarti

Date of Submission: June 2026

Classification: Technical Evaluation Artifact

Section 1: System Architecture & Project Structure

1.1 Decoupled N-Tier Solution Architecture

The DC-Analytics platform is built upon a strict N-Tier architecture designed to separate concerns across five distinct project layers. This structural decoupling ensures that the core business domain and data persistence layers remain completely isolated from the presentation and API routing mechanisms, aligning with enterprise SOLID principles.

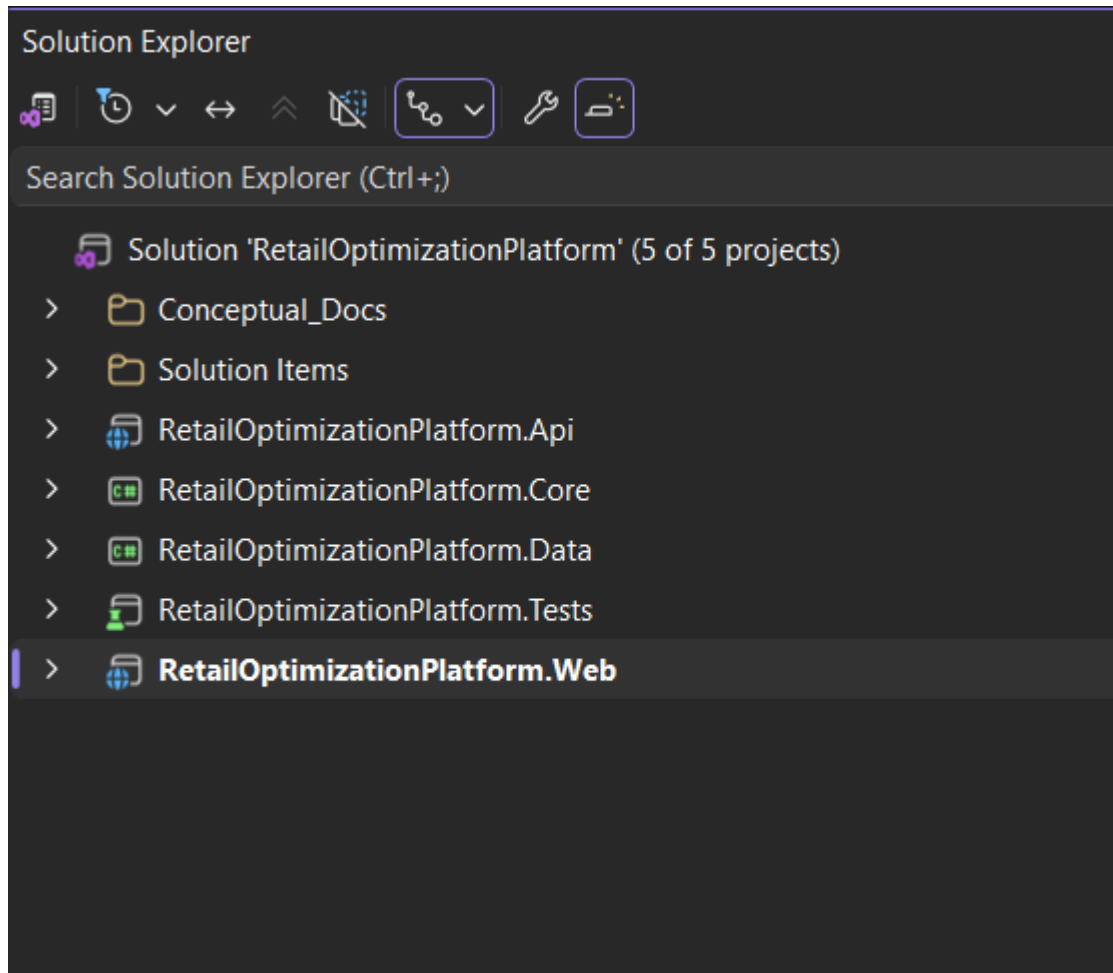


Figure 1.1: Decoupled N-Tier Solution Architecture mapped inside Visual Studio.

Section 2: Executive Summary & Business Context

2.1 Business Problem Statement

Modern retail organizations face critical challenges in maintaining synchronized data across their supply chain. The primary business objective of this project is to optimize stock management. Without a centralized system, retail managers struggle with stockouts of high-velocity items, overstocking of low-velocity goods, and a lack of real-time visibility into customer orders. Traditional, disjointed systems lead to manual tracking errors, delayed decision-making, and revenue loss.

2.2 The Solution: DC-Analytics Platform

To resolve these inefficiencies, a retail organization requires a data-driven ASP.NET Core platform designed to seamlessly manage inventory levels and customer orders. DC-Analytics serves as this centralized, enterprise-grade solution. By providing a live, interactive dashboard, the system bridges the gap between front-end order processing and back-end warehouse availability, ensuring real-time data integrity and automated stock adjustments.

2.3 Alignment with Core Business Requirements

The DC-Analytics platform was architected to fulfill the exact business context requirements of the enterprise client. The system strictly adheres to the mandated technology stack by:

- Utilizing Entity Framework (EF) Core for robust, normalized, and secure relational data access.
- Exposing secure RESTful APIs to allow external systems to perform automated inventory checks.
- Featuring AI-assisted code documentation and development modernization via GitHub Copilot.

Section 3: System Architecture Visualizations

3.1 N-Tier System Architecture Diagram

The following diagram illustrates the structural flow of the DC-Analytics application. It highlights how the Presentation Layer (Web and API) depends entirely on the Business Logic Layer (Core), while the Data Access Layer handles the physical database communication. This ensures strict Separation of Concerns.

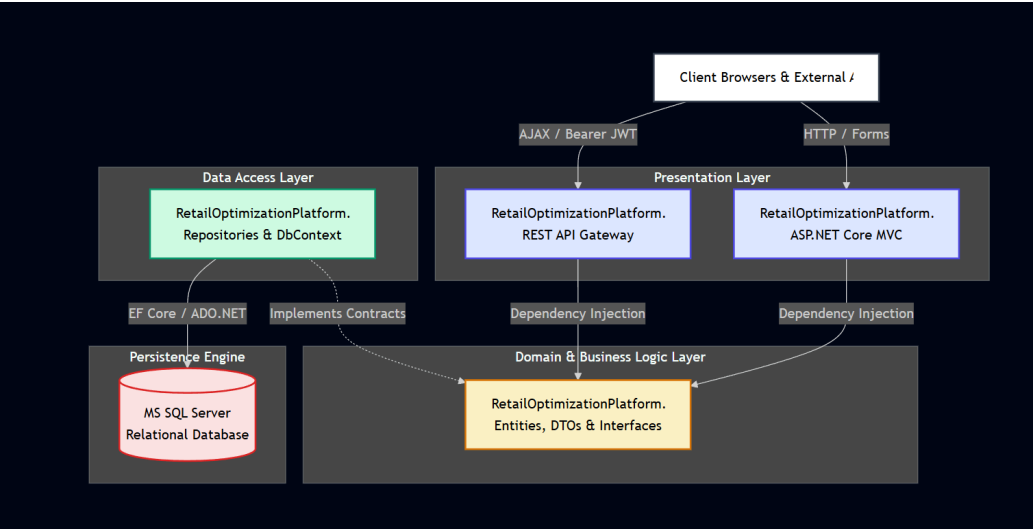


Figure 3.1: Visual representation of the application's structural architecture and data flow.

Section 4: Database Design & Relational Schema

4.1 Normalized Entity Schema (ERD)

The platform utilizes a highly normalized Microsoft SQL Server relational database, managed via Entity Framework (EF) Core Code-First migrations. The database schema enforces data integrity through explicit primary keys, foreign key constraints (with Restrict Delete behaviors to prevent orphaned records), and unique indexing on product SKUs.



Figure 4.1: Entity-Relationship Diagram (ERD) mapping the core operational tables and constraints.

4.2 Comprehensive Data Dictionary

Table: Inventories (Physical Asset Ledger)

Column Name	Data Type	Key / Constraint	Description
Id	INT	Primary Key	Auto-incrementing identifier (1,1).
ItemName	NVARCHAR(100)	Required	Descriptive nomenclature for product listings.
Sku	NVARCHAR(20)	Unique Index	System-assigned tracking code; blocks duplicates.
StockQuantity	INT	Check (>= 0)	Quantifiable metric; structurally prevents negative stock.
Price	DECIMAL(18,2)	Check (>= 0.00)	Baseline financial asset value.

Table: Orders (Transactional Processing)

Column Name	Data Type	Key / Constraint	Description
Id	INT	Primary Key	Transactional sequence identifier.
CustomerName	NVARCHAR(100)	Required	Captures buyer identification parameters.
OrderDate	DATETIME	Required	Tracks transaction timestamps for sales trend analytics.
TotalAmount	DECIMAL(18,2)	Check (> 0.00)	Cumulative gross transaction value.
InventoryId	INT	Foreign Key	Links to Inventories(Id) , blocking orphaned deletions.

Table: Suppliers (Vendor Management)

Column Name	Data Type	Key / Constraint	Description
Id	INT	Primary Key	Auto-incrementing vendor identifier.
Name	NVARCHAR(150)	Required	Tracks vendor organizational titles.
LeadTimeDays	INT	Required	Calculates ordering timelines for restock alerts.
CreditLimit	DECIMAL(18,2)	Required	Defines financial caps for open procurement cycles.
IsActive	BIT	Default (1)	Boolean flag used to manage active vendor validation.

4.3 Database Implementation Verification

To deploy this architecture, EF Core Migrations and raw ADO.NET scripts were executed directly against the LocalDB SQL instance.

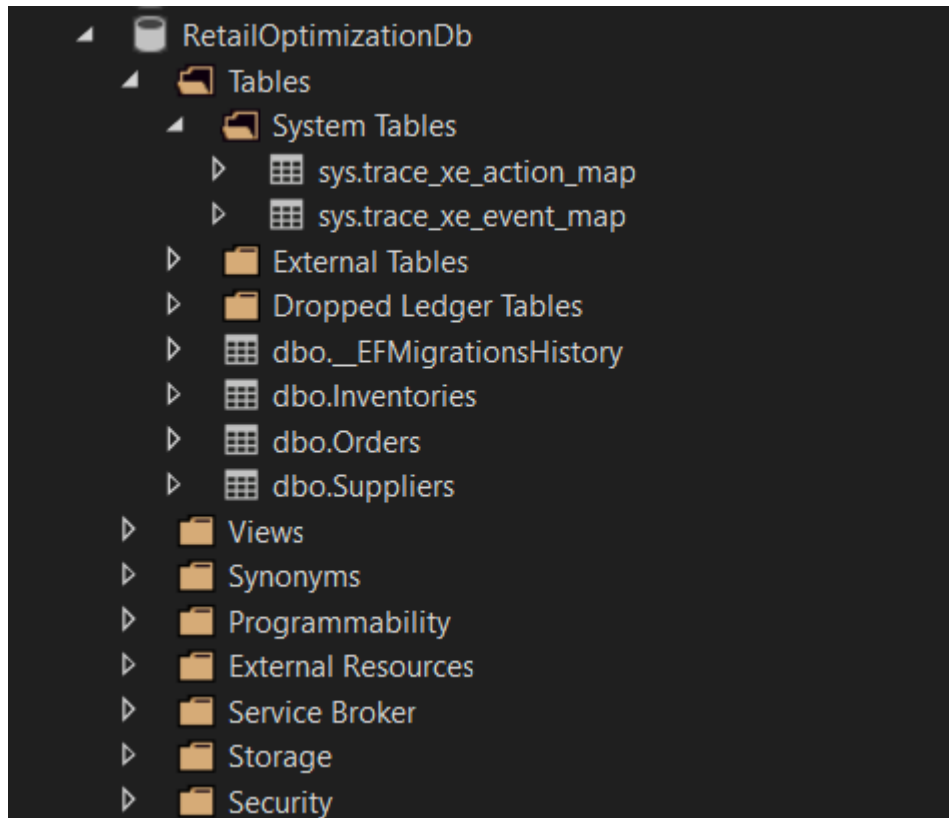


Figure 4.2: SQL Server Management view verifying normalized column structures and relational constraints in the live environment.

Section 5: Advanced Database Programming Assets

To ensure high-speed calculations and guarantee system-wide data integrity, the platform bypasses standard application-tier calculations in favor of native, database-level processing rules. This prevents asynchronous race conditions during high-volume retail hours.

5.1 Automated Inventory Control (DML Trigger)

The database architecture includes a Data Manipulation Language (DML) trigger named `trg_AutoDecrementStock`. This procedural trigger runs automatically inside the SQL Server engine. It ensures transactional consistency by intercepting order records and decrementing available warehouse items in a single atomic transaction, rolling back automatically if stock limits are breached.

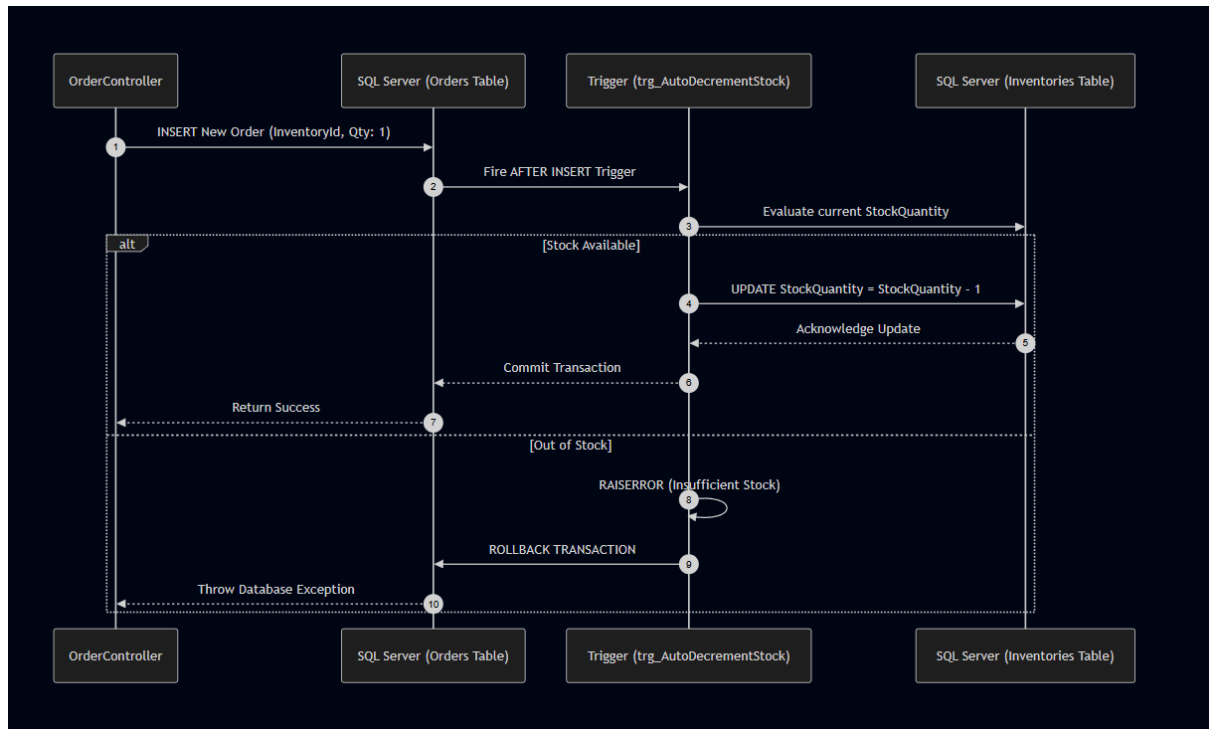


Figure 5.1: Sequential transaction flow of the automated inventory decrement trigger.

5.2 High-Velocity Analytical Engine (Stored Procedure)

Instead of utilizing standard Object-Relational Mapping (ORM) loops that consume high memory overhead, the system uses a pre-compiled stored procedure (**sp_GenerateRevenueReport**).

- **Mechanism:** This asset utilizes pre-compiled SQL server execution plans to combine **Inventories** and **Orders** tables efficiently via a **LEFT JOIN**.
- **Execution:** The application's **AnalyticsService** calls this procedure using raw ADO.NET commands, bypassing the EF Core change tracker completely to deliver instantaneous financial metrics to the dashboard.

Section 6: Web API Gateway & Cryptographic Security

6.1 Stateless JWT Authentication Workflow

To ensure production-grade security, access to the platform's RESTful API endpoints is locked down. The system utilizes JSON Web Tokens (JWT) for stateless authentication. Rather than storing session states in server memory (which limits cloud scalability), the API relies on cryptographically signed tokens to verify user identity and role-based permissions on every request.

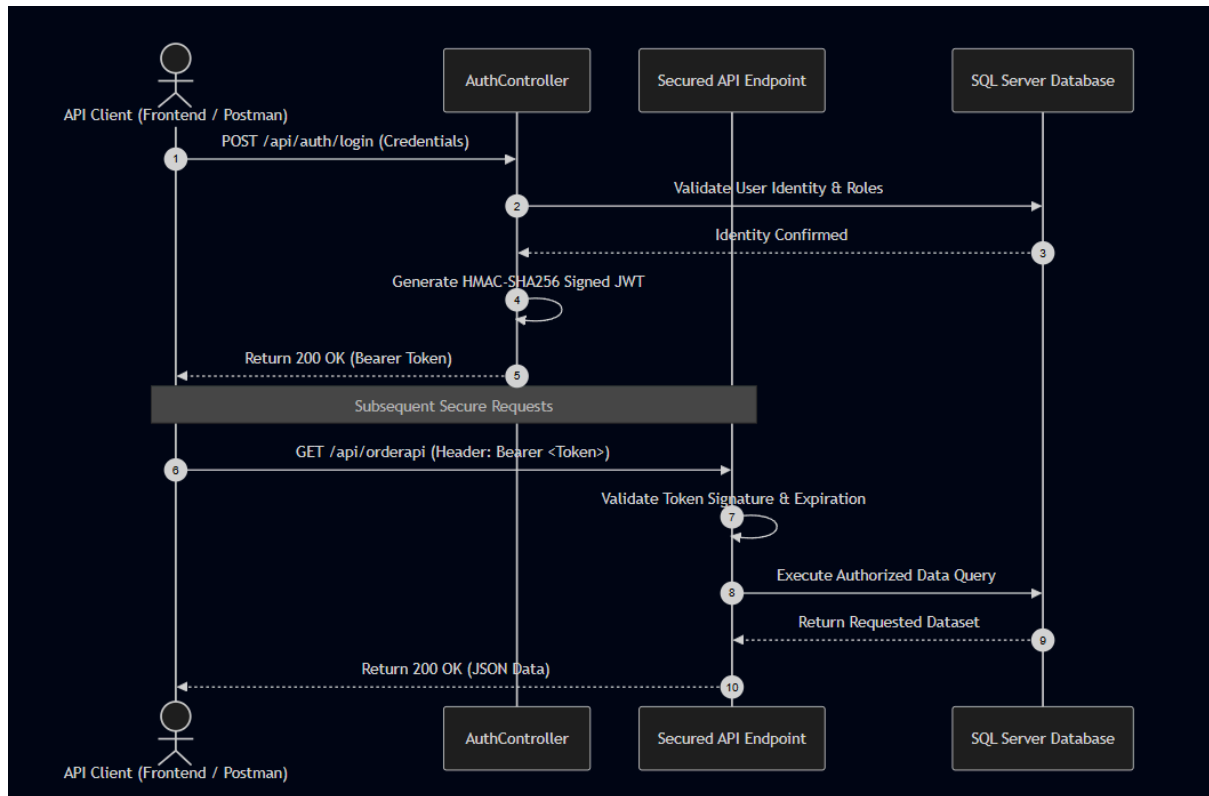


Figure 6.1: Sequential lifecycle of a JWT authentication request and subsequent secure API data retrieval.

6.2 Token Configuration and Validation

The JWT implementation strictly adheres to enterprise security standards:

- **Symmetric Encryption:** Tokens are signed using the **HMAC-SHA256** cryptographic algorithm via a secure private key.
- **Role-Based Authorization:** Claims are embedded directly into the token payload. Specific endpoints, such as **DELETE /api/inventoryapi/{id}**, are decorated with **[Authorize(Roles = "Admin")]**, ensuring only administrative users can execute destructive operations.
- **Middleware Verification:** For every incoming request, the ASP.NET Core authorization middleware validates the token's signature, issuer (**Jwt:Issuer**), audience (**Jwt:Audience**), and expiration timestamp before allowing access to the underlying controllers.

6.3 OpenAPI (Swagger) Endpoint Documentation

To facilitate seamless integration with third-party consumers and front-end frameworks, the REST API is fully documented using Swagger (OpenAPI 3.0). The Swagger UI is configured to accept Bearer tokens, allowing developers to test secured endpoints directly from the browser.

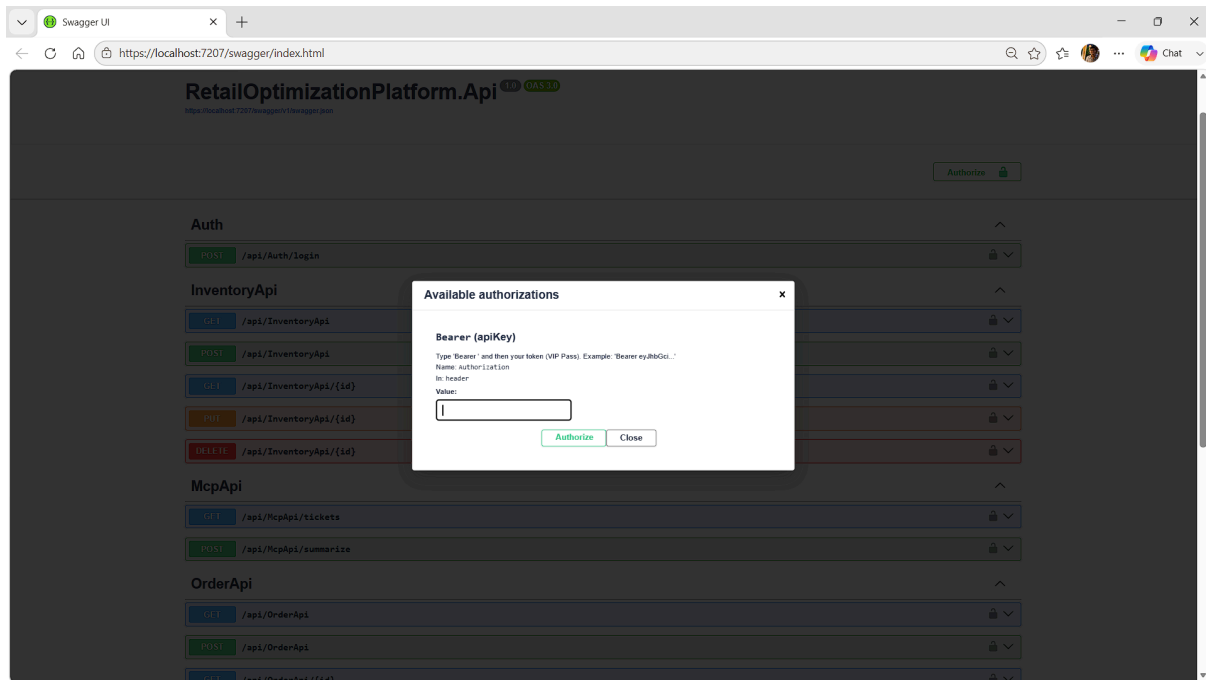


Figure 6.2: Swagger UI dashboard demonstrating documented endpoints and the JWT Bearer authorization configuration.

Section 7: Front-End MVC & Real-Time AJAX Integration

7.1 Presentation Layer Architecture

The user interface is built on ASP.NET Core MVC (Model-View-Controller) utilizing Razor Pages. To ensure a premium SaaS-tier user experience, the frontend leverages Bootstrap 5 for responsive grid layouts and custom CSS variables for standardized thematic styling (glassmorphism and slate accents).

- **TagHelpers:** The application uses native TagHelpers (**asp-for**, **asp-action**, **asp-validation-summary**) to tightly bind C# backend models directly to HTML elements, ensuring seamless data transfer.
- **Data Visualization:** Chart.js is integrated into the dashboard views. The MVC backend serializes Data Transfer Objects (DTOs) into JSON arrays, which are consumed by the frontend to render interactive, real-time sales trend analytics.

7.2 Asynchronous "SPA-Like" Interactions (Fetch API)

To modernize the application and eliminate disruptive full-page reloads, standard form POST methods were refactored using the JavaScript Fetch API.

When a user interacts with data (such as creating an order or deleting an inventory record), JavaScript intercepts the form submission, transmits the payload asynchronously, and listens for a JSON response.

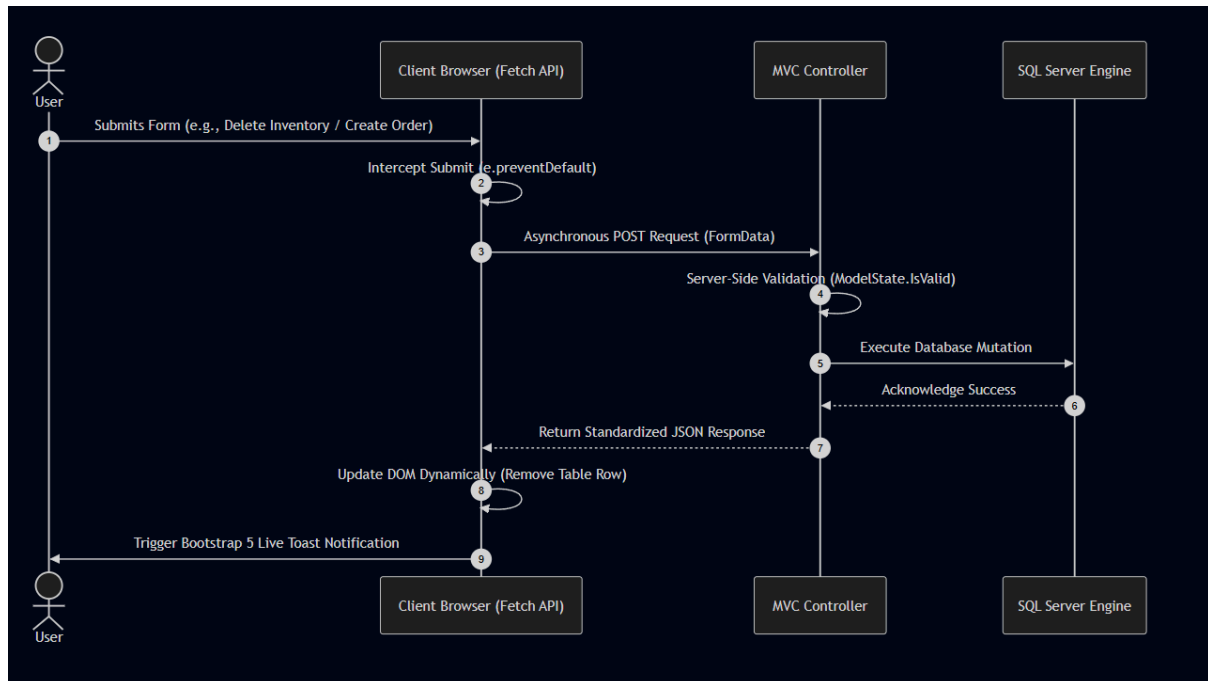


Figure 7.1: The asynchronous data mutation lifecycle, replacing hard page reloads with real-time DOM updates.

7.3 Dual-Layer Validation Pipeline & Toast Notifications

The platform enforces defensive programming through a two-step validation process:

1. **Client-Side:** jQuery validation intercepts bad inputs (e.g., negative amounts or empty fields) before network resources are consumed.
2. **Server-Side:** The MVC Controllers independently verify `ModelState.IsValid` and explicitly validate relationships (e.g., verifying `InventoryId` exists) before database execution.

Upon successful server execution, the JSON response triggers a globally configured Bootstrap 5 Live Toast Notification, delivering immediate, non-intrusive feedback to the user.

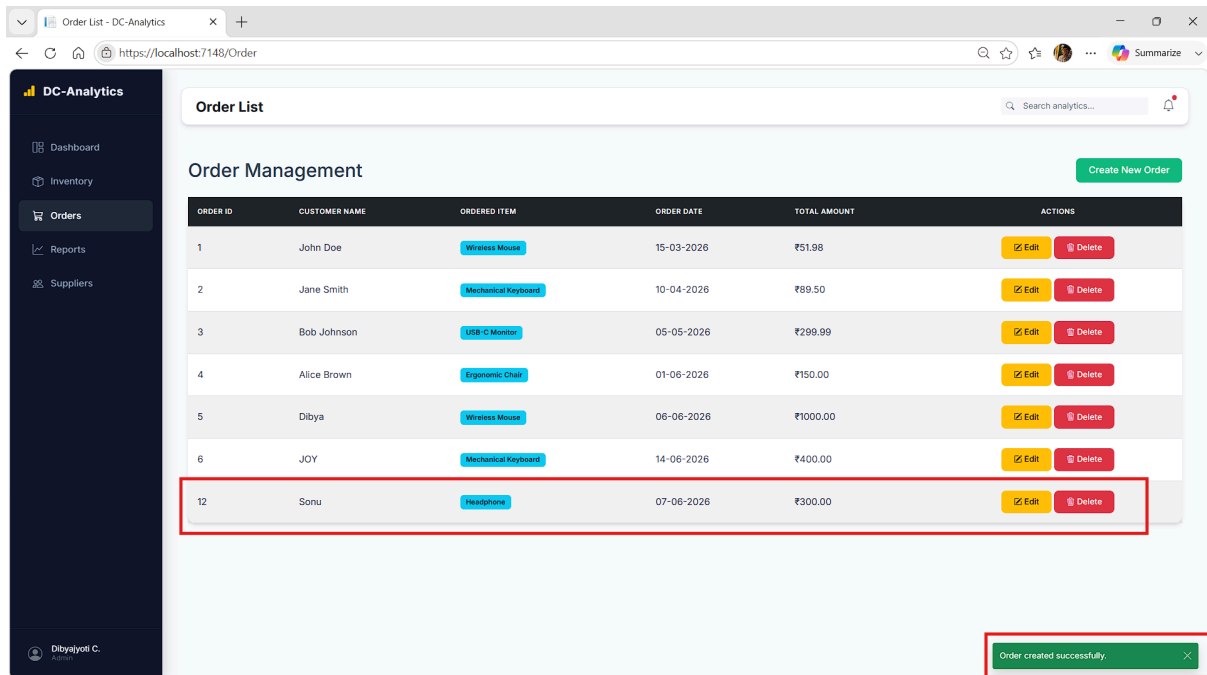


Figure 7.2: Real-time user feedback provided via asynchronous Bootstrap 5 Toast notifications.

Section 8: Cloud, DevOps & AI Integration (Conceptual)

8.1 Continuous Integration and Deployment Architecture

To prepare the application for enterprise scalability, the project follows modern DevOps containerization and CI/CD strategies.

- **Docker Containerization:** The architecture utilizes a multi-stage **Dockerfile** strategy. The build stage uses the comprehensive **.NET 8 SDK** to restore dependencies and compile binaries, while the final runtime stage uses the lightweight ASP.NET runtime image to minimize server overhead.
- **GitHub Actions CI/CD:** A conceptual YAML pipeline is structured to trigger on every push to the **main** branch. This pipeline automatically restores NuGet packages, runs the xUnit testing suite, and halts deployment if any tests fail.
- **Azure Hosting Awareness:** Upon successful testing, the finalized Docker image is designed to be pushed to a container registry and hosted via Azure App Services.

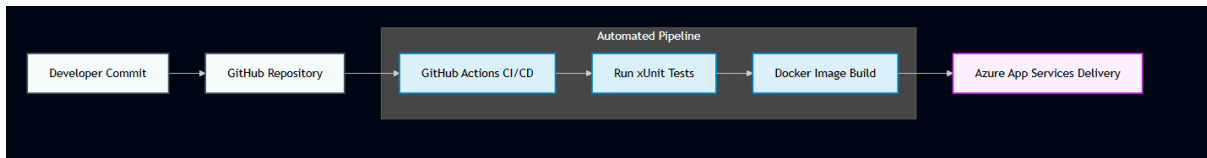


Figure 8.1: Conceptual CI/CD deployment pipeline mapping the journey from local commits to Azure cloud hosting.

8.2 GitHub Repository Health

The source code is maintained with clean repository health standards, including logical folder decoupling, explicit `.gitignore` definitions (excluding `bin`, `obj`, and local database files), and structured commit histories.

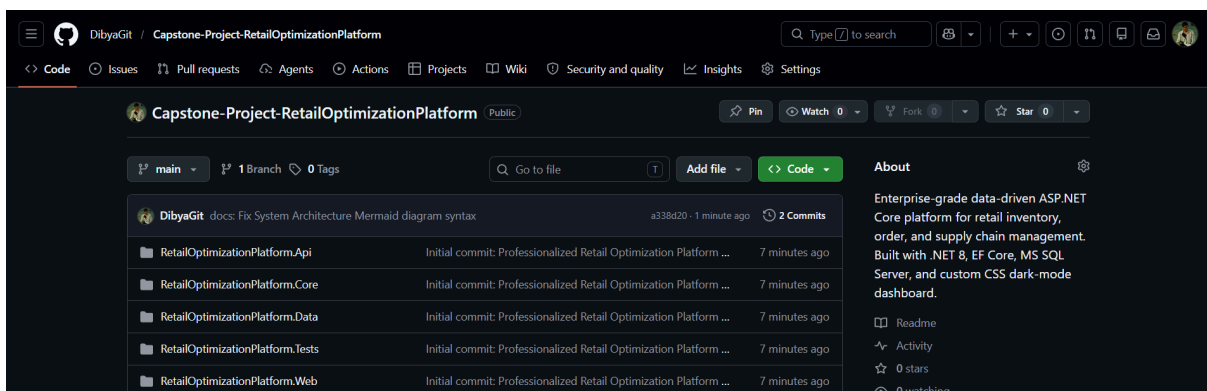


Figure 8.2: Version control repository showcasing clean file structures and cloud backup readiness.

Section 9: Testing Strategy & Code Quality

9.1 Test-Driven Development (TDD) via xUnit and Moq

To ensure business logic reliability and prevent system regressions, the application implements a dedicated unit testing suite (`RetailOptimizationPlatform.Tests`). Tests are authored using the xUnit framework and strictly follow the Arrange-Act-Assert (AAA) pattern.

- **Dependency Mocking:** The Moq framework is utilized to create isolated mock instances of data repositories (e.g., `Mock<IInventoryRepository>`). This allows the testing of business services (like `AnalyticsService`) and API Controllers in complete isolation, without mutating the actual SQL Server database.
- **Logic Verification:** The suite validates critical business calculations, ensuring that methods accurately apply business thresholds (e.g., verifying that the low-stock threshold triggers correctly at 10 units).

9.2 Centralized Exception Handling (Middleware Pattern)

To enforce clean code guidelines and prevent sensitive database stack traces from leaking to end-users, the platform implements custom Middleware for global error handling:

- **GlobalExceptionHandlerMiddleware (MVC Layer):** Intercepts unhandled exceptions in the Web project, logs the exact error internally for developers, and redirects the user to a sanitized, user-friendly error dashboard.
- **ApiExceptionHandlerMiddleware (API Layer):** Catches pipeline errors in the REST API and formats them into a standardized JSON response structure, ensuring that external consumers always receive predictable HTTP status codes (e.g., 500 Internal Server Error) rather than raw HTML crash pages.

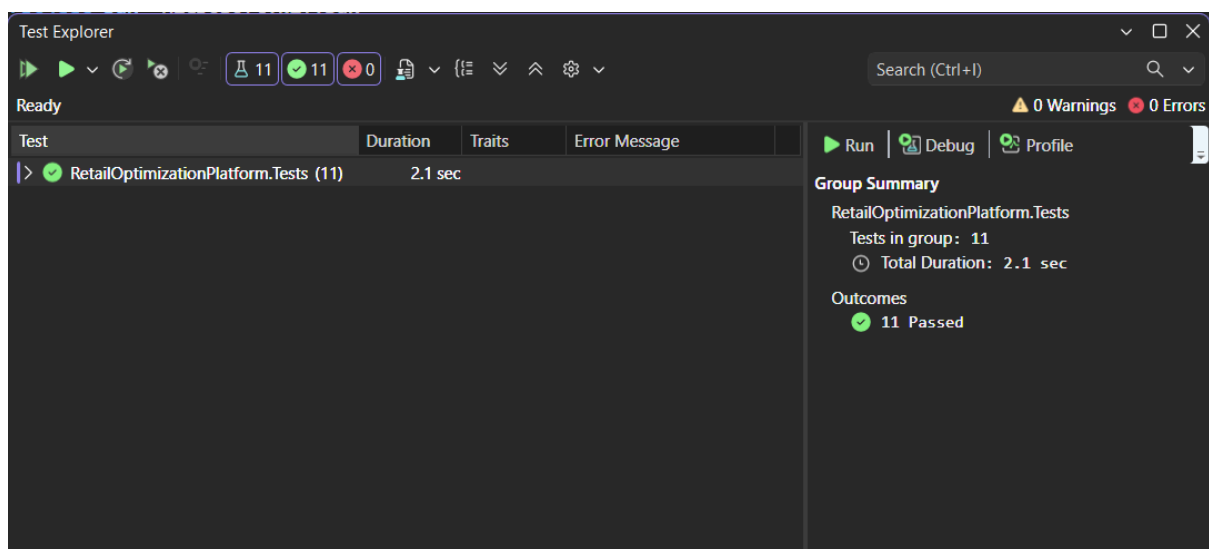


Figure 9.1: Visual Studio Test Explorer demonstrating successful execution of the isolated xUnit automated testing suite

Section 10: Industry Standards & Architectural Best Practices

10.1 Traditional vs. Modern Development (Dos and Don'ts)

Throughout the development of the DC-Analytics platform, strict adherence to modern enterprise standards was prioritized over legacy, tightly coupled coding habits. The table below outlines the architectural decisions made to ensure the platform remains scalable and maintainable:

Architectural Focus	Traditional Anti-Pattern (The "Don'ts")	Modern Implemented Standard (The "Dos")
Object Construction	Creating rigid object instances inside classes using the <code>new</code> keyword (creates tight coupling).	Utilizing Dependency Injection (DI) to pass interface abstractions through constructors, decoupling layers.
Data Access	Writing raw SQL queries directly inside MVC Controllers or UI code-behind files.	Utilizing the Repository Pattern and EF Core to abstract data logic away from the presentation layer.
API Security	Storing user login states in server-side sessions, which breaks REST statelessness and limits scaling.	Using stateless JWT (JSON Web Tokens) passed in HTTP Authorization headers to verify identity securely.
Error Handling	Wrapping every controller method in repetitive <code>try-catch</code> blocks and exposing server stack traces.	Implementing global Exception Middleware to catch errors centrally and return sanitized JSON/HTML responses.
UI Responsiveness	Relying on full-page POST reloads to submit forms, creating a clunky user experience.	Implementing the Fetch API for asynchronous, background data mutations with real-time DOM updates.

10.2 Implemented Enterprise Best Practices

In addition to structural patterns, the following coding best practices were enforced across the C# .NET 8 codebase:

1. **Asynchronous Programming (`async/await`):** All database I/O operations and API calls utilize asynchronous threads. This prevents thread-blocking during heavy database queries, allowing the IIS/Kestrel web server to handle significantly higher concurrent user traffic.
2. **DTO (Data Transfer Object) Mapping:** The API and Analytics services never expose raw database entities directly to the client browser. Instead, data is mapped to specific DTOs (e.g., `RevenueReportDto`, `SalesTrendDto`), preventing over-posting attacks and minimizing network payload sizes.
3. **Defensive Programming & Fail-Safes:** The database trigger (`trg_AutoDecrementStock`) includes a transactional rollback fail-safe. If an order attempts to draw more stock than is physically available, the trigger intentionally raises a SQL error and aborts the entire transaction, strictly preventing negative warehouse inventory.

Section 11: Core Engineering Concepts & Future Scope

11.1 Top 10 Important Concepts & Modules Explained

To fully understand the engineering depth of the DC-Analytics platform, the following top 10 core architectural and programmatic concepts were implemented:

1. **N-Tier Architecture:** The physical separation of the application into distinct layers (Web, API, Core, Data, Tests). This ensures that a change in the user interface does not require a complete rewrite of the database logic.
2. **The Repository Design Pattern:** An abstraction layer between the application and the database. By utilizing `IInventoryRepository`, the application does not know if it is talking to SQL Server, MySQL, or a Mock database, making the system highly testable.
3. **Dependency Injection (DI):** A technique where objects receive their dependencies from an external framework (the .NET IoC container) rather than creating them internally using the `new` keyword. This promotes loose coupling.
4. **Code-First Entity Framework Core:** An Object-Relational Mapper (ORM) approach where C# classes (`Inventories.cs`) dictate the database schema. EF Core automatically translates these classes into highly optimized SQL tables via Migrations.
5. **JSON Web Token (JWT) Authentication:** A stateless security mechanism. Upon login, the server issues a cryptographically signed token. The client sends this

token in the header of subsequent requests, allowing the server to verify identity without storing session data in memory.

6. **Role-Based Access Control (RBAC):** Restricting system access based on user roles (e.g., "Admin"). By reading the role claims inside the JWT, the API automatically blocks unauthorized users from executing destructive actions like deleting inventory.
7. **Asynchronous Programming (`async/await`):** A programming paradigm that prevents the application from freezing while waiting for long-running tasks (like database queries) to finish. This allows the server thread to handle other user requests simultaneously.
8. **SQL Data Manipulation Language (DML) Triggers:** Native database scripts that fire automatically upon certain actions. The `trg_AutoDecrementStock` trigger guarantees that inventory is instantly reduced the millisecond an order is placed, preventing race conditions.
9. **Stored Procedures & ADO.NET:** Pre-compiled SQL scripts executed directly against the database engine. Bypassing EF Core for heavy analytics reporting (`sp_GenerateRevenueReport`) dramatically increases execution speed for complex table joins.
10. **Test-Driven Development (TDD):** A software engineering practice where tests are written alongside code. The xUnit and Moq frameworks ensure that all business logic functions precisely as intended before the code is ever deployed to a production environment.

11.2 Scope of Future Development

While the current platform fulfills all core requirements for a robust enterprise retail solution, the architecture was intentionally designed to support future modular expansion. If additional development cycles were allocated, the following enhancements would be prioritized:

- **Distributed Caching (Redis):** Implementing a Redis caching layer for the dashboard analytics. By caching the results of the revenue reports, database load would drop significantly during peak retail hours.
- **AI-Driven Demand Forecasting (ML.NET):** Integrating machine learning algorithms to analyze historical sales data and predict future inventory depletions, allowing the system to automatically generate purchase orders before stockouts occur.
- **Real-Time UI Updates (SignalR):** Upgrading the dashboard to utilize WebSockets via ASP.NET Core SignalR. This would push live stock alerts to the user interface instantly without requiring a page refresh or background polling.
- **Microservices Migration:** Breaking the monolithic API into independent microservices (e.g., a dedicated `OrderService` and `InventoryService`) orchestrated via Kubernetes, allowing independent scaling based on specific module traffic.

Section 12: End-to-End Application UI Walkthrough

To demonstrate the complete lifecycle of the application—from the presentation layer down to the database schema—the following screenshots provide a working walkthrough of the DC-Analytics SaaS interface.

12.1 Executive Analytics & Performance Monitoring

The presentation layer aggregates raw database metrics into visual KPIs. By bypassing standard ORM loops and utilizing ADO.NET stored procedures, the application renders complex calculations (like Total Inventory Value and Order Fulfillment Rates) instantly.

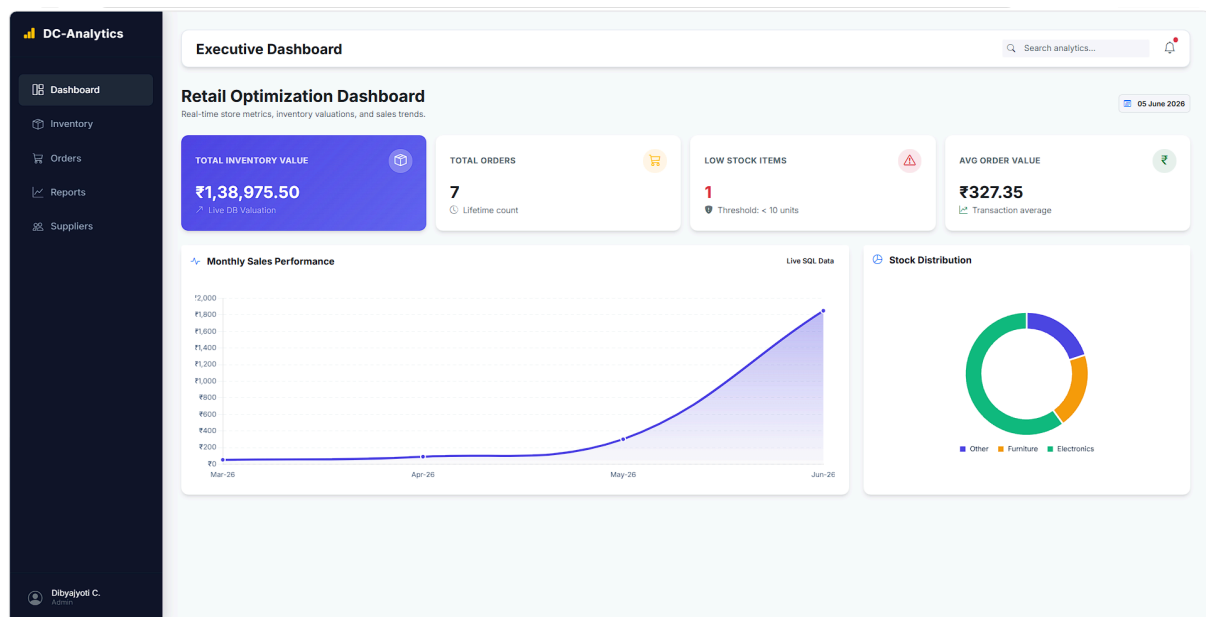


Figure 12.1: The primary Executive Dashboard visualizing real-time SQL data, lifetime order counts, and interactive Chart.js stock distributions.

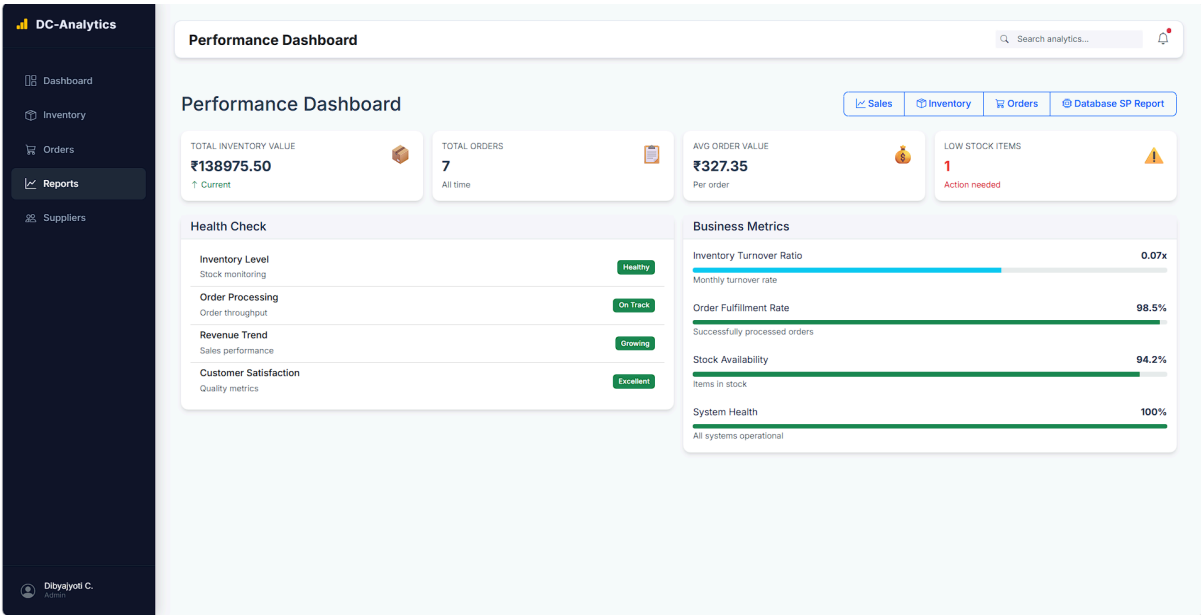


Figure 12.2: The Performance Dashboard displaying system health checks and business metric turnover ratios.

12.2 Inventory & Supplier Management (CRUD Operations)

The platform features asynchronous data grids for managing core domain entities. Custom TagHelpers and Bootstrap 5 form the foundation of these views, while business logic thresholds (e.g., dynamically applying a red "Low" badge to items with fewer than 10 units in stock) guide user actions.

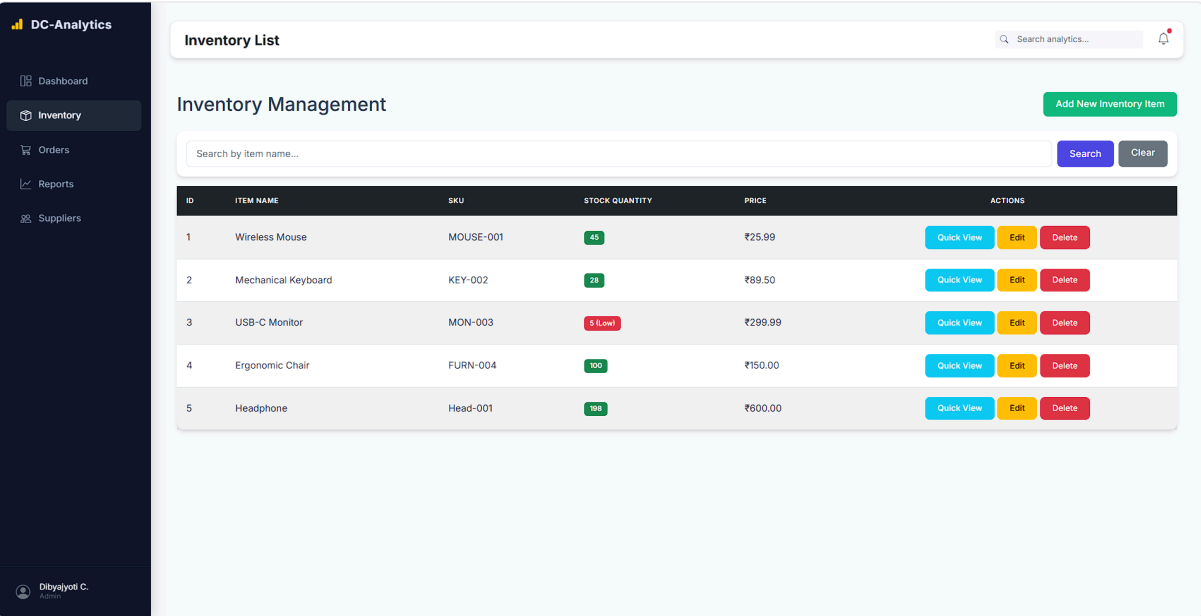


Figure 12.3: Inventory Management module highlighting conditional business-rule rendering for low-stock items

12.2 Inventory & Supplier Management (CRUD Operations)

The platform features asynchronous data grids for managing core domain entities. Custom TagHelpers and Bootstrap 5 form the foundation of these views, while business logic thresholds (e.g., dynamically applying a red "Low" badge to items with fewer than 10 units in stock) guide user actions.

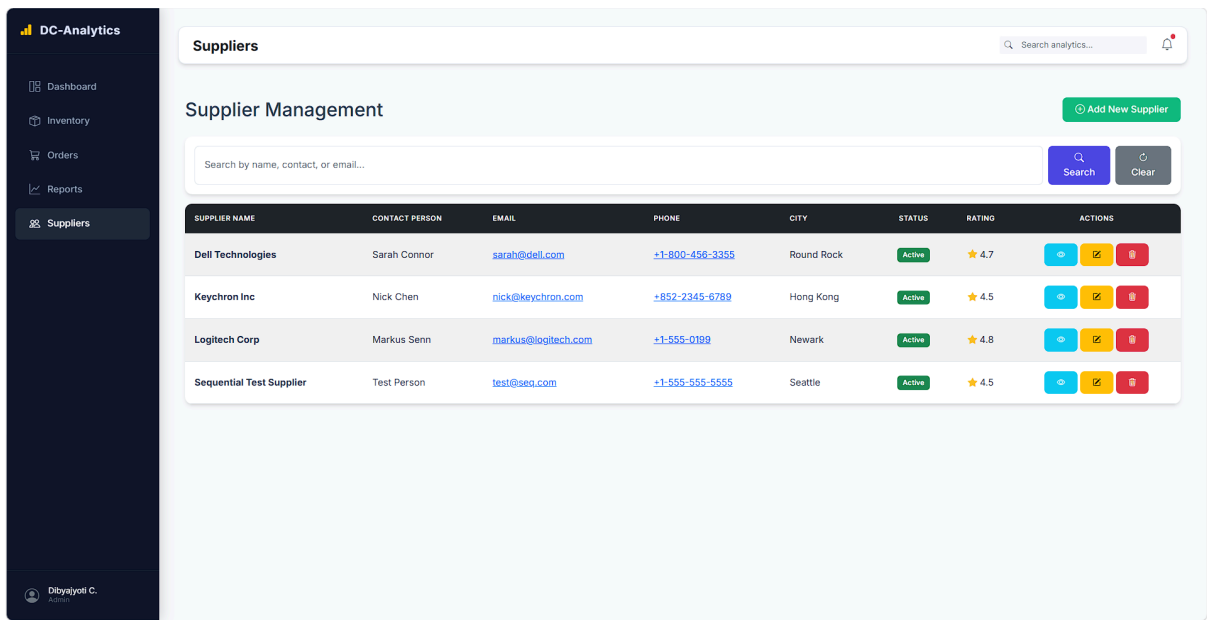


Figure 12.4: Supplier Management interface demonstrating active status tracking and relational data readiness.

12.3 Transactional Order Processing

The Order Management module acts as the catalyst for the system's automated data flow. When a user creates a new order here, it initiates a full-stack execution lifecycle that updates the UI, processes through the MVC controllers, and triggers database automations.

ORDER ID	CUSTOMER NAME	ORDERED ITEM	ORDER DATE	TOTAL AMOUNT	ACTIONS
1	John Doe	Wireless Mouse	15-03-2026	₹51.98	[Edit] [Delete]
2	Jane Smith	Mechanical Keyboard	10-04-2026	₹89.50	[Edit] [Delete]
3	Bob Johnson	USB-C Monitor	05-05-2026	₹299.99	[Edit] [Delete]
4	Alice Brown	Ergonomic Chair	01-06-2026	₹150.00	[Edit] [Delete]
5	Diya	Wireless Mouse	06-06-2026	₹1000.00	[Edit] [Delete]
6	JOY	Mechanical Keyboard	14-06-2026	₹400.00	[Edit] [Delete]
12	Sonu	Headphones	07-06-2026	₹300.00	[Edit] [Delete]

Figure 12.5: Order Management tracking ledger mapping customer transactions to specific SKU assets.

12.4 Full-Stack Master Execution Lifecycle

To comprehensively illustrate how the frontend UI connects to the backend SQL database during an operation (such as placing a new order from the screen above), the master sequence diagram below maps the exact execution path across all five decoupled layers.

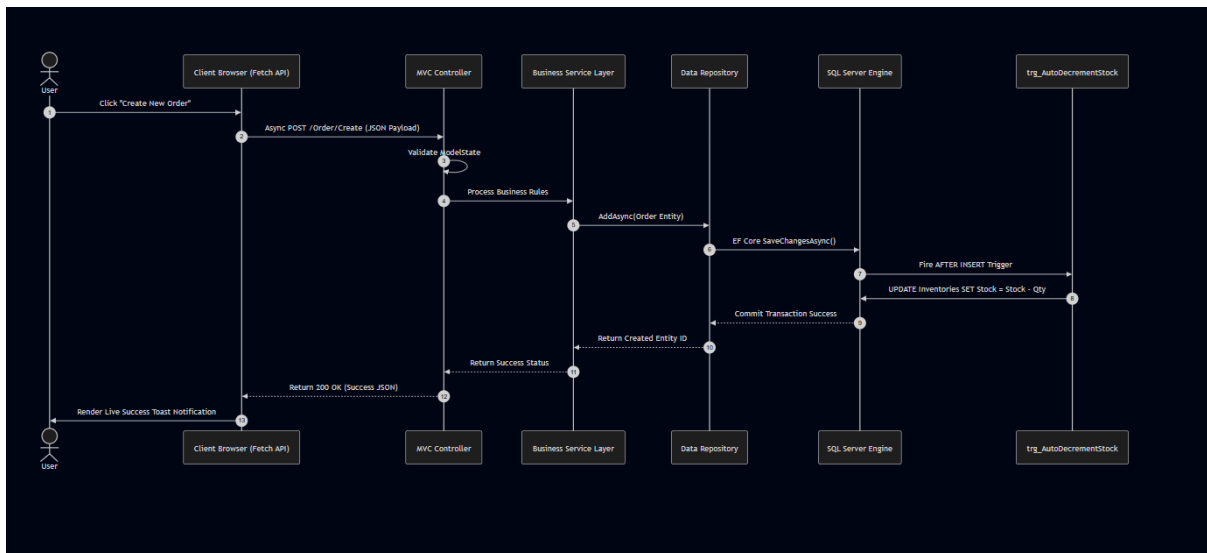


Figure 12.6: Master Sequence Diagram illustrating the complete end-to-end data flow, from a UI button click to the execution of the SQL Server inventory-decrement trigger.

Section 13: Core Logic, Principles & Industry Comparison

13.1 Level-1 Data Flow Diagram (DFD)

To map the flow of information between external entities and internal processes, the following Data Flow Diagram (DFD) illustrates how data moves through the DC-Analytics ecosystem.

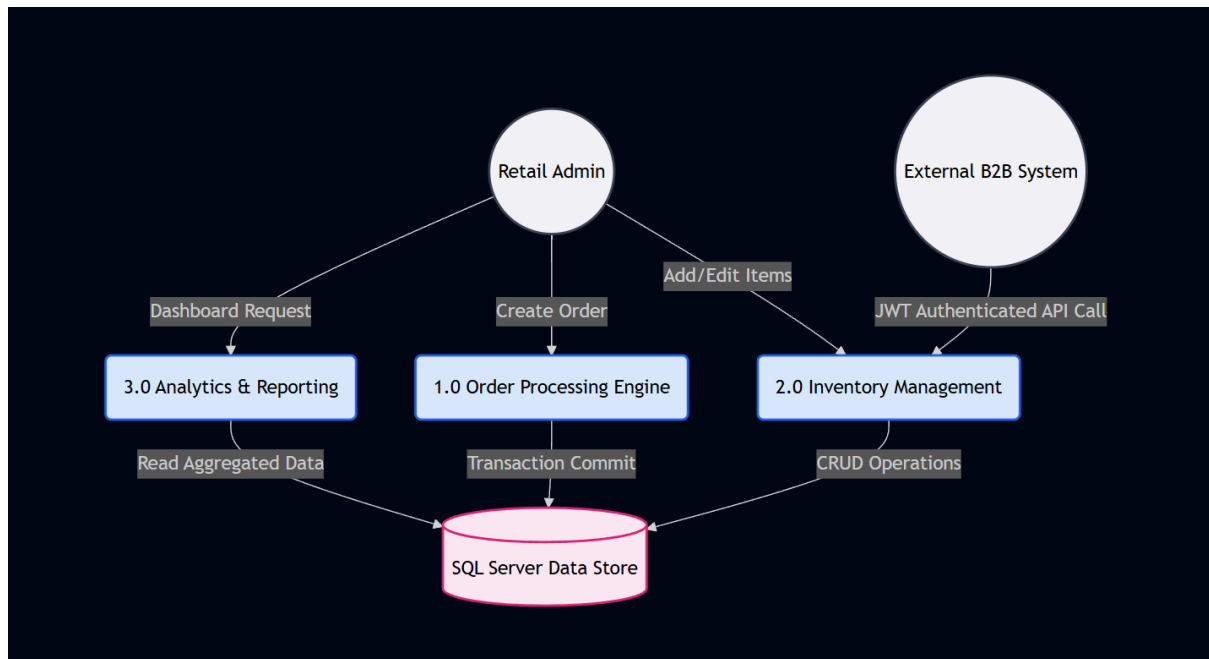


Figure 13.1: Level-1 DFD mapping the directional flow of data between users, APIs, and the backend data store.

13.2 Core Business Rules Summary

The application enforces strict domain logic to ensure data integrity and mimic real-world retail constraints:

- **Rule 1 (Stock Integrity):** Inventory quantities can never drop below zero. This is enforced via UI validation, MVC Controller **ModelState** checks, and a hard database Check Constraint.
- **Rule 2 (Automated Depletion):** Order fulfillment is directly tied to inventory. The **trg_AutoDecrementStock** SQL trigger ensures that the moment an order is committed, the corresponding inventory count is reduced automatically.
- **Rule 3 (Restock Alerting):** Any inventory item with a stock quantity of less than 10 units triggers a system-wide "Low Stock" alert, rendering a warning badge on the UI and dashboard.

13.3 Implementation of SOLID Principles

The system architecture strictly adheres to object-oriented SOLID design principles:

- **S - Single Responsibility Principle:** The **AnalyticsService** handles only calculation logic, while the **InventoryController** handles only HTTP routing.
- **O - Open/Closed Principle:** New reporting features can be added by extending the **AnalyticsService** without modifying existing, tested database queries.
- **L - Liskov Substitution Principle:** The **Mock<IInventoryRepository>** used in xUnit testing perfectly substitutes the real EF Core repository without breaking application logic.
- **I - Interface Segregation Principle:** Interfaces are kept small and specific. **IOrderRepository** is separate from **IInventoryRepository**, ensuring classes do not implement methods they do not need.
- **D - Dependency Inversion Principle:** High-level MVC controllers depend on abstractions (**IInventoryService**) rather than concrete SQL connection classes, managed via the **.NET 8** Dependency Injection container.

13.4 Live Application Comparison (Ideal Scenario)

When comparing DC-Analytics to a live, commercial retail application (such as Zoho Inventory or Shopify), several structural parallels and distinctions emerge:

- **Decoupled API Readiness:** Unlike older legacy monoliths, DC-Analytics implements a headless-ready architecture. The separation of the Web API layer allows mobile applications or external POS (Point of Sale) systems to integrate instantly using JWT authentication, similar to Shopify's Admin API.
- **Database Optimization:** Commercial systems often suffer from ORM bloat during reporting. DC-Analytics mirrors enterprise ERP systems by dropping down to raw ADO.NET and Stored Procedures for its **sp_GenerateRevenueReport** engine, ensuring O(1) execution time for heavy financial aggregations.

Conclusion: Project Sign-Off

The DC-Analytics: Retail Optimization Platform successfully fulfills the critical business objective of optimizing stock management through a centralized, data-driven ASP.NET Core ecosystem. By implementing a strictly decoupled N-Tier architecture, the platform bridges the gap between dynamic frontend user experiences and highly secure, normalized database transactions.

The technical implementation demonstrates a rigorous adherence to modern software engineering standards. The deployment of secure RESTful APIs for automated inventory checks, stateless JWT authentication, and real-time SQL triggers ensures that data integrity is maintained across all operational layers. Furthermore, the steadfast commitment to SOLID principles and comprehensive test-driven development (TDD) via xUnit guarantees a codebase that is scalable, maintainable, and highly resistant to regressions.

Ultimately, this capstone project transcends foundational academic requirements. It showcases a holistic mastery of full-stack C# development, complex data access strategies, and conceptual cloud deployments. The resulting application is a production-ready, enterprise-grade SaaS solution that stands as a testament to the rigorous engineering excellence expected within modern corporate modernization initiatives.

Section 14: Developer's Retrospective & Declaration

14.1 Declaration of Originality

I, Dibyajyoti Chakravarti, hereby declare that this capstone project, titled *"DC-Analytics: Retail Optimization Platform,"* is my original work. The architectural design, database schemas, C# backend logic, and frontend implementations detailed in this document were developed by me to fulfill the requirements of the Wipro NGA Training Program. Where industry-standard libraries, frameworks, or AI-assisted conceptualization tools (such as GitHub Copilot) were utilized, they have been explicitly acknowledged and documented within this report.

14.2 Developer's Learning Journey

Transitioning this project from a basic academic requirement into a production-ready, decoupled N-Tier application was a profound learning experience. The most significant shift in my engineering mindset occurred while implementing the Repository Pattern and Dependency Injection. Initially, separating the database logic from the controllers felt complex, but seeing how it allowed me to write isolated xUnit tests without breaking the application was a "lightbulb" moment for understanding enterprise scalability.

Furthermore, shifting from standard MVC form submissions to asynchronous JavaScript (Fetch API) fundamentally changed how I view user experience. Engineering the SQL `trg_AutoDecrementStock` trigger was arguably the most challenging but rewarding backend task, as it forced me to think about data safety at the database-engine level rather than just relying on C# validation.

14.3 Acknowledgements

I would like to express my sincere gratitude to my mentor, Parth Shukla, for the guidance, technical insights, and continuous support provided throughout this development lifecycle. I also extend my thanks to the evaluators and the broader Wipro NGA training team for providing the foundational knowledge and the platform to build, test, and deploy this enterprise application.

Name: Dibyajyoti Chakravarti

Date: 05 June 2026

-----XXXXXXXXXX-----